Annexure-I

Data Structure & Algorithm Using CPP
Centre of Professional Enhancement,LPU

A training report

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

(Computer Science and Engineering)

Submitted to

LOVELY PROFESSIONAL UNIVERSITY
PHAGWARA, PUNJAB

From June 10, 2025 to July 18, 2025

SUBMITTED BY

Name of student: Vivek Kumar

Registration Number: 12305176

Annexure-II: Student Declaration

To whom so ever it may concern

I, **Vivek Kumar**, Registration Number **[12305176]**, hereby declare that the work done by me on **"Data Structure & Algorithm using CPP"** from **June, 2025** to **July, 2025**, is a record of original work for the partial fulfilment of the requirements for the award of the degree, **B.Tech. in Computer Science and Engineering**.

Vivek Kumar(12305176)

Dated: 28/08/2025

ACKNOWLEDGEMENT

Primarily I would like to thank God for being able to learn a new technology. Then I would like to express my special thanks of gratitude to the teacher and instructor of the course **Data Structure & Algorithm using CPP** from Centre of Professional Enhancement who provide me the golden opportunity to learn a new technology from home.

I would like to also thank my own college Lovely Professional University for offering such a course which not only improve my programming skill but also taught me other new technology.

Then I would like to thank my parents and friends who have helped me with their valuable suggestions and guidance for choosing this course.

Last but not least I would like to thank my all classmates who have helped me a lot.

Date: 28/08/2025

Vivek Kumar
12305176

Summer Training Certificate

CENTRE FOR
PROFESSIONAL
ENHANCEMENT

Certificate No. 407013

NAAC A++

Certificate of Merit

This is to certify that Mr./Ms. Vivek Kumar S/D/W/o Mr. Ashok Kumar Singh
student of School of Computer Science and Engineering Registration No. 12305176
pursuing Bachelor of Technology (Computer Science and Engineering) completed
skill development course named Data Structures and Algorithms using C++
organized by Centre for Professional Enhancement Lovely Professional University
from 10 June 2025 to 18 July 2025 and obtained B Grade.

Date of Issue : 13-08-2025
Place of Issue: Phagwara (India)

Prepared by
(Administrative Officer-Records)

Programme Coordinator
Centre for Professional Enhancement

Head of School
School of Computer Science and Engineering

Table of Contents

S. No.	Title	Page
1	Cover page	1
2	Declaration of the student	2
3	Acknowledgement	3
4	Training Certification from organization	4
5	Table of Contents	5
6	List of abbreviation	6
7	Introduction	7
8	Technology learnt	10
9	Project code	27
10	Project output	31
11	Introduction of the Project Undertaken	33
12	Introduction of The Company/Work	36
13	Brief Description of The Work Done	37
14	Conclusion	40
15	References	42

Abbreviation

- **DSA:** Data Structures and Algorithms.
- **C++:** C Plus Plus (Programming Language)

INTRODUCTION

The term DSA stands for Data Structures and Algorithms. In this context, our focus is on learning and implementing them using C++, a powerful, high-performance programming language widely used in competitive programming, system-level applications, and software development.

C++ is chosen because it offers:

- **Speed and efficiency** close to low-level languages like C
- **Object-Oriented Programming (OOP)** for better structure and modularity
- **Rich Standard Template Library (STL)** that provides ready-to-use implementations of common data structures and algorithms
- **Fine control over memory management** through pointers, dynamic allocation, and RAII (Resource Acquisition Is Initialization)

What is a Data Structure?

A **Data Structure** is a systematic way of storing and organizing data so that it can be accessed and modified efficiently. In C++, data structures can be implemented either **manually** (e.g., creating your own linked list class) or **using STL containers** like vector, map, queue, etc.

For example, in C++:

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Player {
    string name;
    int age;
};

int main() {
    Player p1 = {"Virat", 26};
    cout << "Name: " << p1.name << ", Age: " << p1.age;
    return 0;
}
```

Here, "Virat" is stored as a string (sequence of characters) and 26 as an int (integer data type). Organizing such data in a struct or class makes it easy to manage related information together.

Key Idea: Choosing the right data structure (array, linked list, stack, queue, tree, graph, etc.) is critical for writing efficient C++ programs.

What is an Algorithm?

An **Algorithm** is a finite sequence of well-defined instructions to solve a problem. In C++, algorithms are implemented as functions or programs that take some input and produce the desired output.

An algorithm is **not** the entire program—it's the core **logic** of the solution. You can express an algorithm:

- In **pseudocode**
- As a **flowchart**
- Directly in **C++ code**

Example – **Finding Maximum in an Array (Algorithm in C++):**

```cpp
#include <iostream>
using namespace std;

int findMax(int arr[], int n) {
    int maxVal = arr[0];
    for(int i = 1; i < n; i++) {
        if(arr[i] > maxVal) {
            maxVal = arr[i];
        }
    }
    return maxVal;
}

int main() {
    int arr[] = {5, 8, 2, 10, 3};
    int n = 5;
    cout << "Maximum element: " << findMax(arr, n);
}
```

Complexity: O(n) time, O(1) space.

About the Course: DSA using C++

This training covered **Data Structures and Algorithms from basic to advanced levels**, entirely with **C++ implementations**.

The course was designed over **weeks** of structured learning, but being **self-paced**, I could progress at my own speed.

- Each week focused on a different set of topics like arrays, recursion, sorting, graphs, dynamic programming, etc.
- Practical coding problems and quizzes tested my understanding.
- Implementation was done in **pure C++** as well as using **STL**.

TECHNOLOGY LEARNT

During my training, I learned Data Structures and Algorithms completely in C++ — starting from the basics and moving to advanced topics.

Here's what I covered:

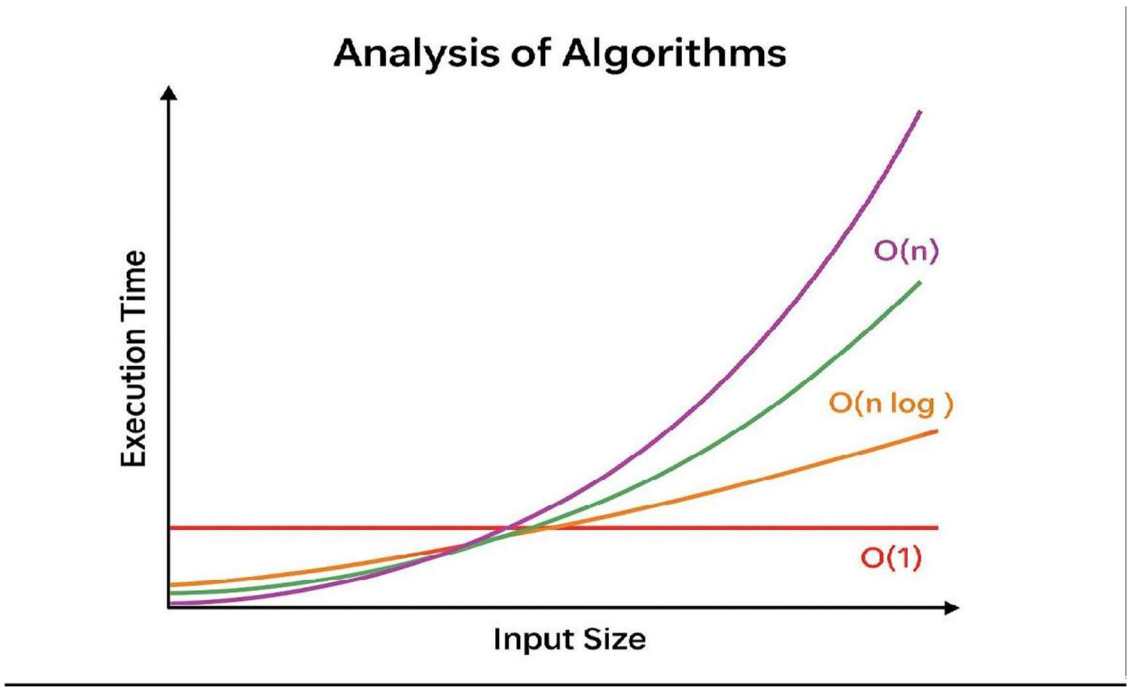
1. Analysis of Algorithms

Understanding how algorithms behave with different input sizes is the first step in DSA. We measure Time Complexity (execution speed) and Space Complexity (memory usage).

Types of Performance:

- Best case → Fastest possible execution
- Worst case → Slowest possible execution
- Average case → Expected runtime for typical inputs

Graph – Growth Rates of Time Complexities:



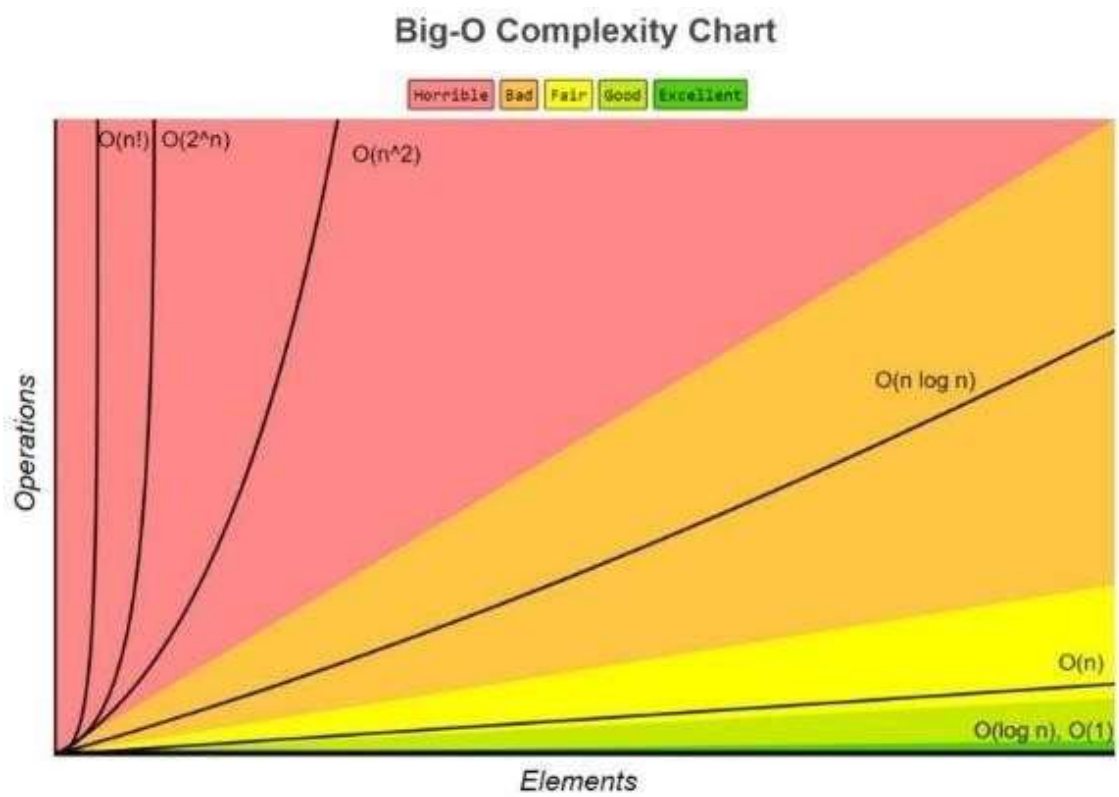
2. Order of Growth

Order of growth shows how performance changes with n (input size).

Example:

- O(1) → Accessing first element in array
- O(n) → Linear Search
- O(n log n) → Merge Sort
- O(n^2) → Nested loops for comparisons

Graph – Order of Growth Comparison:



3. Asymptotic Notations

These are mathematical tools to describe algorithm performance.

1. Big O (O) – Upper bound (worst-case)
2. Omega (Ω) – Lower bound (best-case)
3. Theta (Θ) – Tight bound (average-case)

4. Big O Example in C++ – Linear Search

```cpp
#include <iostream>
using namespace std;

int linearSearch(int arr[], int n, int key) {
    for(int i = 0; i < n; i++)
        if(arr[i] == key) return i;
    return -1;
}

int main() {
    int arr[] = {5, 10, 15, 20};
    int n = 4;
    cout << "Index: " << linearSearch(arr, n, 15);
}
```

Time Complexity: O(n) worst case
Space Complexity: O(1)

5. Analysis of Common Loops

Single loop: O(n)
Nested loops: $O(n^2)$
Independent loops: O(n + m)

Example – Nested Loops in C++

```
for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
        cout << i << "," << j << "\n";
```

6. Recursion & Recurrence Relations

Recursion is a function calling itself.
Performance is measured with **recurrence relations** like:
T(n) = T(n-1) + O(1) (factorial)
T(n) = T(n-1) + T(n-2) + O(1) (Fibonacci)

Example – Factorial

```
int factorial(int n) {
    if(n == 0) return 1;
    return n * factorial(n - 1);
}
```

7. Space Complexity

Space complexity includes:

- Fixed memory usage (constants, variables)
- Dynamic memory (arrays, structures)
- Function call stack (recursion)

Example – Recursive Space Usage:
Factorial recursion → O(n) stack frames.

8. Mathematics for DSA in C++

Mathematics is essential in many algorithm problems:

- Prime number testing
- GCD & LCM
- Modular arithmetic
- Combinatorics

Example – GCD

```
int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}
```

9. Bit Manipulation in C++

Bit manipulation is the process of performing operations directly on the binary representation of data.
It is widely used in optimization problems, cryptography, and low-level programming.

Key Bitwise Operators in C++:

- AND (&)
- OR (|)
- XOR (^)
- Left Shift (<<)
- Right Shift (>>)
- Bitwise NOT (~)

Example – Check Kth Bit is Set

```
bool checkKthBit(int n, int k) {
    return (n & (1 << k)) != 0;
}
```

10. Arrays in C++

An **array** is a collection of elements stored at contiguous memory locations.
C++ arrays can be **static** (fixed size) or **dynamic** (allocated at runtime using new).

Advantages:

- Direct access to elements (O(1) time)
- Easy iteration

Limitations:

- Fixed size (for static arrays)
- Insertion/deletion is costly

Example – Reverse an Array

```
void reverseArray(int arr[], int n) {
    int start = 0, end = n - 1;
    while(start < end) swap(arr[start++], arr[end--]);
}
```

11. Searching Algorithms

Searching finds the position of a given element in a collection.

Linear Search – O(n) time, simple but slow for large datasets.
Binary Search – O(log n) time, works on sorted arrays.

Example – Binary Search

```
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while(low <= high) {
        int mid = low + (high - low) / 2;
        if(arr[mid] == key) return mid;
        else if(arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

12. Sorting Algorithms

Sorting arranges elements in a specific order.

Common Sorting Algorithms in C++:

- Insertion Sort – $O(n^2)$
- Merge Sort – O(n log n)
- Quick Sort – O(n log n) average
- STL sort() – O(n log n) (Introsort)

Example – Using STL sort

```
#include <algorithm>
sort(arr, arr + n);
```

13. Matrix in C++

A matrix is a 2D array used for storing data in rows and columns.

Common Matrix Operations:

- Transpose
- Rotation
- Spiral traversal
- Multiplication

Example – Transpose

```
for(int i = 0; i < n; i++)
    for(int j = i; j < m; j++)
        swap(mat[i][j], mat[j][i]);
```

14. Hashing in C++

Hashing maps data to fixed-size values for **fast access**.

C++ STL Hash Containers:

- unordered_map – Key-value storage
- unordered_set – Unique elements

Advantages:

- Average O(1) search and insert

Example – Word Frequency

```
unordered_map<string, int> freq;
freq["apple"]++;
```

15. Strings in C++

Strings store sequences of characters.
In C++, std::string from STL is widely used.

Common Operations:

- Concatenation (+)
- Substring extraction (substr)
- Search (find)

Example – Palindrome Check

```
bool isPalindrome(string s) {
    int l = 0, r = s.size() - 1;
    while(l < r) if(s[l++] != s[r--]) return false;
    return true;
}
```

16. Linked List (Singly, Doubly, Circular)

A **Linked List** is a linear data structure where elements (nodes) are connected using pointers instead of being stored in contiguous memory.

Types:

1. **Singly Linked List** – Each node points to the next node.
2. **Doubly Linked List** – Each node has pointers to both next and previous nodes.
3. **Circular Linked List** – Last node points back to the first node.

Advantages:

- Dynamic memory allocation
- Easy insertion/deletion

Disadvantages:

- No direct access to elements
- Extra memory for pointers

Example – Node Structure in C++

```
struct Node {
    int data;
    Node* next;
};
```

17. Stack in C++

A **Stack** is a **LIFO (Last In First Out)** data structure.
Insertion and deletion happen at the same end (top).

Applications:

- Undo/redo functionality
- Expression evaluation

- Function call management

Example – Stack using STL

```
#include <stack>
stack<int> s;
s.push(10);
s.pop();
```

18. Queue & Deque in C++

A **Queue** is a **FIFO (First In First Out)** data structure.

- **Front** → Deletion
- **Rear** → Insertion

A **Deque (Double-ended Queue)** allows insertion and deletion from both ends.

Applications:

- Scheduling
- Buffer management
- Sliding window problems

Example – Queue using STL

```
#include <queue>
queue<int> q;
q.push(5);
q.pop();
```

19. Trees in C++

A **Tree** is a hierarchical structure with nodes connected by edges.
The **root node** is the starting point, and each node may have child nodes.

Binary Tree: Each node has at most two children.

Tree Traversals:

1. **Inorder** (Left, Root, Right)
2. **Preorder** (Root, Left, Right)
3. **Postorder** (Left, Right, Root)
4. **Level Order** (Breadth-First)

20. Binary Search Tree (BST)

A **BST** is a binary tree where:

- Left child < Parent
- Right child > Parent

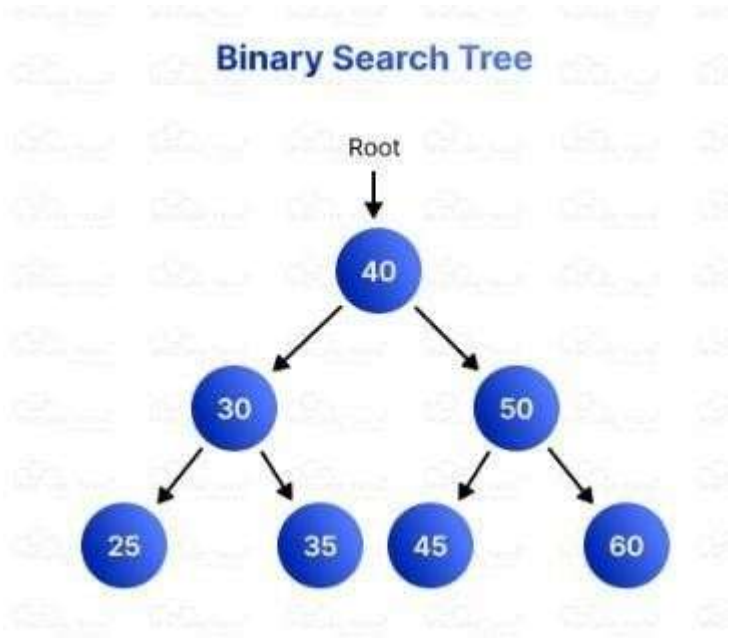
Advantages:

- Efficient search, insertion, deletion (O(log n) average)

Example – Search in BST

```
bool search(Node* root, int key) {
    if(!root) return false;
    if(root->data == key) return true;
    if(key < root->data) return search(root->left, key);
    else return search(root->right, key);
}
```

Diagram – BST Structure:



21. Heap & Priority Queue in C++

A **Heap** is a complete binary tree with heap property:

- **Max Heap:** Parent ≥ children
- **Min Heap:** Parent ≤ children

Priority Queue in STL uses heap internally for efficient retrieval of highest/lowest priority.

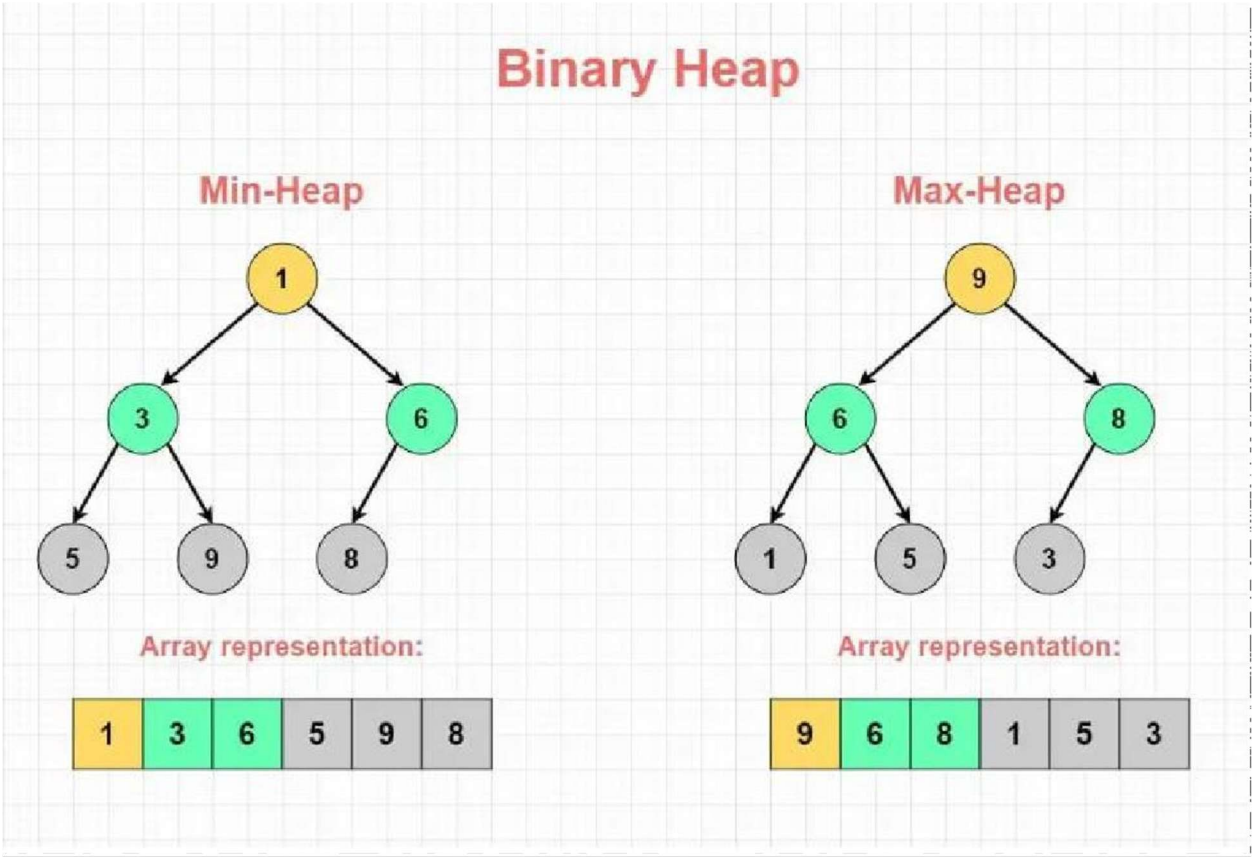
Applications:

- Heap sort
- Dijkstra's algorithm
- Scheduling

Example – Min Heap in C++

```
priority_queue<int, vector<int>, greater<int>> pq;
```

Diagram – Heap Tree & Array Representation:



22. Graphs in C++

A **Graph** is a collection of nodes (vertices) connected by edges.
Graphs can be **directed** or **undirected**, **weighted** or **unweighted**.

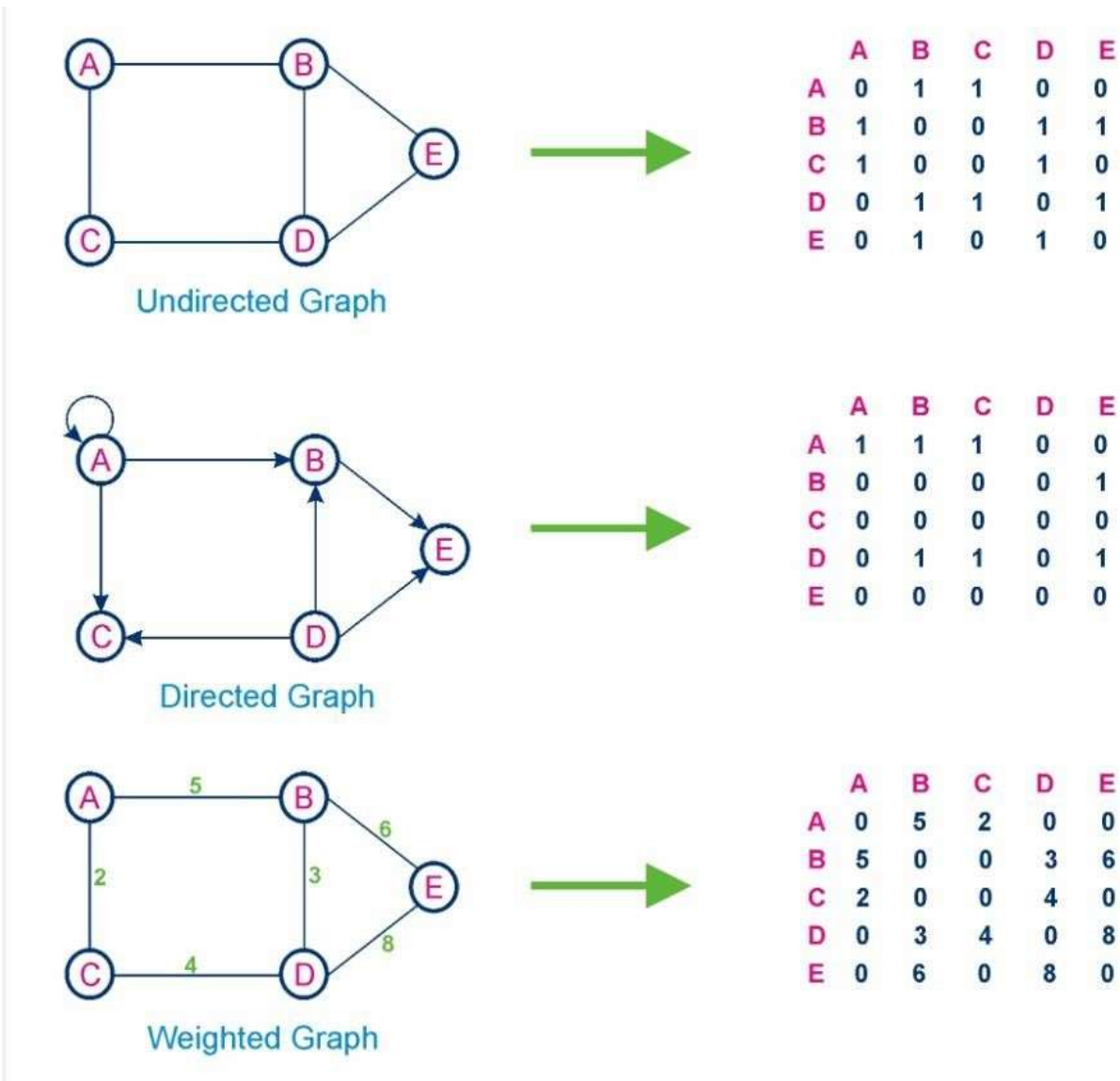
Graph Representations in C++:

1. **Adjacency Matrix** – 2D array storing edge weights ($O(V^2)$ space).
2. **Adjacency List** – Vector of lists storing neighbours ($O(V + E)$ space).

Common Graph Algorithms:

- **BFS (Breadth-First Search)** – Level-wise traversal
- **DFS (Depth-First Search)** – Depth exploration
- **Dijkstra's Algorithm** – Shortest paths
- **Prim's/Kruskal's** – Minimum Spanning Tree

Diagram – Graph Representation:



23. Greedy Algorithms

Greedy algorithms make the locally optimal choice at each step in the hope of finding a global optimum.

Common Greedy Problems in C++:

- Activity Selection
- Fractional Knapsack
- Huffman Coding

Example – Activity Selection Approach:

1. Sort activities by finishing time.
2. Select next activity that starts after last chosen one.

24. Backtracking

Backtracking tries all possible solutions and abandons a path when it's not feasible.

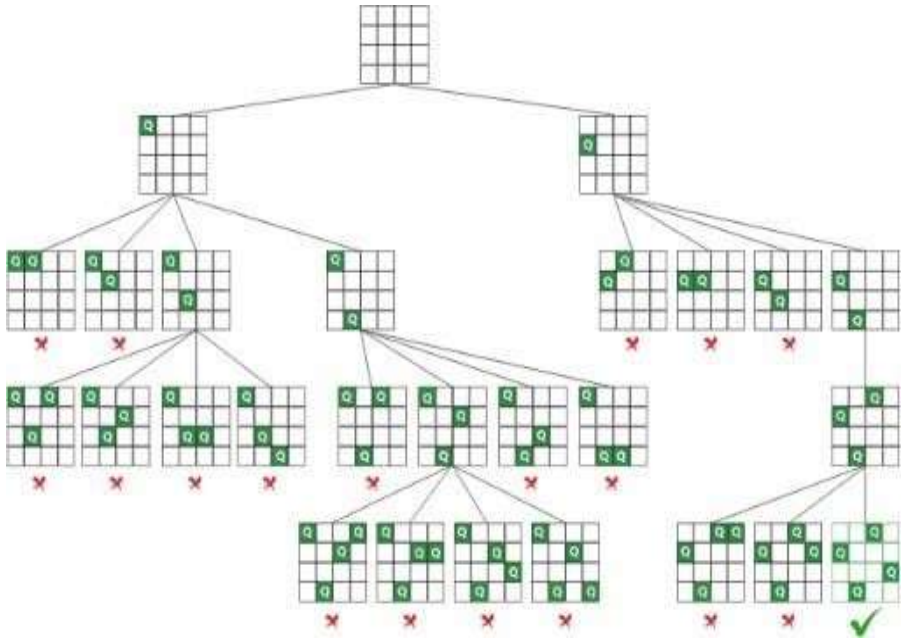
Popular Problems:

- N-Queens
- Rat in a Maze
- Sudoku Solver

Example – N-Queens Idea:

1. Place a queen in a row.
2. Move to the next row and place queen where no conflict occurs.
3. Backtrack if no safe position found.

Diagram – N-Queens Placement:



25. Dynamic Programming (DP)

Dynamic Programming solves problems by breaking them into overlapping subproblems and storing results to avoid recomputation.

Two Approaches:

- **Memoization** (Top-down)
- **Tabulation** (Bottom-up)

Common DP Problems:

- Longest Common Subsequence (LCS)
- 0/1 Knapsack
- Fibonacci numbers

Project Code

```
#include <iostream>
#include <vector>
#include <queue>
#include <climits>
#include <string>

using namespace std;

class RouteOptimizer {
private:
    int numLocations;
    vector<string> locationNames;
    vector<vector<pair<int, int>>> graph;

public:
    RouteOptimizer(int n) {
        numLocations = n;
        graph.resize(n);
        locationNames.resize(n);
    }

    void setLocationName(int index, const string& name) {
        locationNames[index] = name;
    }

    void addRoute(int from, int to, int distance) {
        graph[from].push_back({to, distance});
        graph[to].push_back({from, distance});
```

```
        graph[from].push_back({to, distance});
        graph[to].push_back({from, distance});
    }

    void findShortestPaths(int source) {
        vector<int> dist(numLocations, INT_MAX);
        dist[source] = 0;

        priority_queue<pair<int, int>, vector<pair<int, int>>,
            greater<>> pq;
        pq.push({0, source});

        while (!pq.empty()) {
            int current = pq.top().second;
            int currentDist = pq.top().first;
            pq.pop();

            for (auto& neighbor : graph[current]) {
                int next = neighbor.first;
                int weight = neighbor.second;

                if (dist[current] + weight < dist[next]) {
                    dist[next] = dist[current] + weight;
                    pq.push({dist[next], next});
                }
            }
        }

```

```
            }
        }

        cout << "\n Shortest distances from: " << locationNames[source] << "\n";
        for (int i = 0; i < numLocations; ++i) {
            cout << "To " << locationNames[i] << " = ";
            if (dist[i] == INT_MAX)
                cout << "Unreachable\n";
            else
                cout << dist[i] << " units\n";
        }
    }
};

int main() {
    int n, e;
    cout << "Enter number of locations: ";
    cin >> n;

    RouteOptimizer ro(n);

    cout << "Enter names of locations:\n";
    for (int i = 0; i < n; ++i) {
        string name;
        cout << "Location " << i << ": ";
        cin >> name;
        ro.setLocationName(i, name);
```

```
    cout << "Enter names of locations:\n";
    for (int i = 0; i < n; ++i) {
        string name;
        cout << "Location " << i << ": ";
        cin >> name;
        ro.setLocationName(i, name);
    }

    cout << "\nEnter number of routes (edges): ";
    cin >> e;

    cout << "Enter route details: from to distance (as indices 0 to
        " << n - 1 << ")\n";
    for (int i = 0; i < e; ++i) {
        int from, to, distance;
        cout << "Route " << i + 1 << ": ";
        cin >> from >> to >> distance;
        ro.addRoute(from, to, distance);
    }

    int source;
    cout << "\nEnter source location index: ";
    cin >> source;

    ro.findShortestPaths(source);

    return 0;
}
```

Project Output

```
Enter number of locations: 4
Enter names of locations:
Location 0: A
Location 1: B
Location 2: C
Location 3: D

Enter number of routes (edges): 5
Enter route details: from to distance (as indices 0 to 3)
Route 1: 0 1 10
Route 2: 0 2 15
Route 3: 1 3 12
Route 4: 2 3 10
Route 5: 1 2 5

Enter source location index: 0
```

```
 Shortest distances from: A
To A = 0 units
To B = 10 units
To C = 15 units
To D = 22 units
```

Introduction of the Project Undertaken

Project Title: Hospital Management System Queue

1. Project Objective:

The primary objective of this project is to **develop a Route Optimization System** that computes the **shortest paths between various locations** using **Dijkstra's Algorithm**. This project is designed to help users plan efficient travel routes by minimizing total distance traveled.
The program takes user inputs for:

- The **number of locations** (nodes).
- The **names of the locations** for better readability.
- The **routes between locations** (edges) along with their **distances** (weights).
- A **source location** from which the shortest distances to all other locations are calculated.

The system processes this information and **outputs the shortest distance from the source location to every other location**, or indicates if a location is **unreachable**.
.

2. Scope of Work:

The **Route Optimizer Project** is designed to provide a **simplified yet effective system** for computing the **shortest distances between multiple locations** using graph-based algorithms.
The scope of this project outlines the boundaries, functionalities, and limitations of the system to ensure clarity and well-defined objectives.

1. **Graph Representation of Routes:**
   - Locations are represented as **nodes** and routes as **edges** with associated
   - distances (weights).
   - The system supports a **bidirectional graph**, meaning routes work in both
   - directions.
2. **Shortest Path Calculation:**
   - Uses **Dijkstra's Algorithm** to calculate the **shortest distance** from a selected
   - **source location** to all other locations.
   - Displays distances in a **clear, user-friendly format**.
3. **User Input Support:**
   - Dynamic input for the number of locations, route details, and location names.
   - Allows the user to select a **source location** for distance calculation.
4. **Console-Based Backend:**
   - Provides a **text-based backend interface** that is easy to use and suitable for
   - academic or prototype purposes.

- Designed in **C++** for efficiency and scalability.

5. **Customizability:**
    - The structure is **flexible** to allow future integration with a **frontend**
    - (HTML, CSS, JavaScript) for better visualization.
    - Can be enhanced for **larger networks** like city maps, delivery planning,
    - or navigation systems.

3. Project Description:

The **Route Optimizer** is a console-based application developed in **C++** to calculate the shortest distances between multiple locations. This project utilizes **graph data structures** and implements
Dijkstra's Algorithm to determine optimal routes efficiently.

The system allows users to:

- Input the number of locations and their names.
- Define bidirectional routes between locations with specified distances.
- Choose a source location and view the shortest distances to all other locations.

This project demonstrates the practical use of **priority queues, adjacency lists, and shortestpath algorithms**, making it a great example of applying **graph theory** concepts to solve real-world
routing problems. It can be extended into a full-fledged **navigation or delivery route planning system** with further integration of mapping tools and user-friendly interfaces.

4. System Architecture:

The Route Optimizer system is designed with a modular and layered architecture to ensure clarity, scalability, and efficient execution. It follows a simple three-layered structure:

1. Input Layer
   - This layer is responsible for interacting with the user.
   - The user provides the number of locations, their names, and the distances between them.
   - Inputs are stored in suitable data structures for further processing.
2. Processing Layer
   - This layer contains the RouteOptimizer class, which handles the core computation:
     - Graph Representation: The locations and routes are represented using an adjacency list (vector of pairs).
     - Shortest Path Algorithm: Dijkstra's Algorithm is implemented using a priority queue for efficient distance calculation.
     - Bidirectional Edges: Routes are added in both directions, reflecting real-world roads.
3. Output Layer
   - Displays the computed shortest distances from the selected source location to all other locations.
   - If a location is unreachable, it is clearly marked in the output.

5. Workflow and Step-by-Step Process:

The project follows a sequential workflow from user input to shortest path calculation and result display. Here's the high-level process:

1. Start Program
   - The system initializes and prompts the user to input details.
2. Input Locations
   - The user provides the total number of locations.
   - Each location is named and stored.
3. Input Routes
   - The user specifies the number of routes (edges).
   - For each route, the user enters:
     - Starting location (index)
     - Destination location (index)
     - Distance (weight)
   - The routes are stored in a graph structure (adjacency list).
4. Set Source Location
   - The user selects a starting point for path calculation.
5. Apply Dijkstra's Algorithm
   - The program calculates the shortest distance from the source location to all other locations.
   - A priority queue is used to efficiently select the next closest node.

6. Display Results
    - The system displays shortest distances for all locations:
        - If reachable: Shows the shortest distance.
        - If unreachable: Shows "Unreachable".
7. End Program
    - The user can review results or restart the process for new inputs.

6. Key Features and Highlights:

- **Shortest** Route Calculation
- Implements Dijkstra's Algorithm to calculate the shortest paths between a source location and all other locations.
- Graph-Based System
- The road network is represented as a graph using adjacency lists, ensuring efficient memory usage and fast computation.
- Custom Location Names
- Users can input meaningful names for each location (e.g., "Campus", "Library", "Market"), making the system user-friendly.
- Dynamic Route Input
- Supports dynamic entry of routes between locations, allowing flexibility for various map or city layouts.
- Bidirectional Routes
- Each route is considered two-way, making the system realistic for real-world navigation scenarios.
- Handles Unreachable Locations
- Displays a clear message when a location is unreachable, improving error handling and clarity.
- Efficient Algorithm Implementation
- Uses priority queues for efficient path selection, optimizing the algorithm's performance to O((V+E) log V).
- Simple Command-Line Interface (CLI)
- Provides a step-by-step input-based system, making it lightweight and easy to run on any machine without extra tools.

7. Technologies Used:

- **C++** ensures optimal **time complexity** for route calculation, making it faster than scripting languages for large datasets.
- The use of **Dijkstra's Algorithm** guarantees the **most efficient path calculation** in weighted graphs.
- **STL containers and priority queues** simplify implementation while maintaining high performance.

8. Role and Profile:

As the **Project Developer**, my role was to conceptualize, design, and implement the **Route Optimizer** system. I was responsible for:

- **Analyzing the problem** of determining the shortest and most efficient routes.
- **Designing the system architecture** and workflow for route calculation.
- **Implementing the project in C++**, ensuring efficient use of data structures (graphs, adjacency lists, and priority queues).
- **Applying Dijkstra's Algorithm** to compute optimal routes.
- **Testing and validating** the program for accuracy and handling edge cases.
- **Documenting the entire project** in a structured and professional manner.

9. Learning Outcomes:

- Gained hands-on experience in full-stack development using C++ for backend processing.
- Learned the process of validating, storing, and retrieving dynamic data.
- Understood client-server architecture and real-time web interaction.
- Improved problem-solving and system design skills by integrating front-end and backend technologies.
- Developed skills in deploying web applications on a local server environment.

10. Conclusion:

The **Route Optimizer** project successfully demonstrates how graph theory and algorithms can be applied to solve real-world problems like route planning and shortest path determination. By implementing **Dijkstra's Algorithm** in C++, this project achieves an efficient and scalable way to compute the shortest distances between locations in a network. The program is designed to be **user-friendly**, allowing users to input location names, routes, and distances dynamically. It provides accurate results even for large graphs, making it suitable for applications in **logistics, transportation planning, navigation systems, and delivery services**. The use of **efficient data structures** such as adjacency lists and priority queues ensures minimal memory consumption and fast computation times.

This project also serves as a strong foundation for further development. Features like **visual route mapping, traffic prediction, real-time data integration, and multi-criteria optimization** can be added to make it more practical and commercially viable.
In summary, this project not only demonstrates theoretical knowledge of algorithms and data structures but also highlights their **practical application** in solving real-life challenges. It has strengthened problem-solving, algorithmic thinking, and programming skills while offering a solution with real-world significance.
.

Introduction of The Company/Work

- **Company's Vision and Mission**: Lovely Professional University (LPU) aims to transform education through academic excellence, innovation, and holistic student development. The mission of the Centre for Professional Enhancement (CPE) is to equip students with industry-ready skills, enhance employability, and bridge the gap between classroom knowledge and real-world applications.

- **Origin and growth of company**: The Centre for Professional Enhancement at LPU was established to address the growing need for professional training, career readiness, and soft skills development among students
- **Various departments and their functions**: The Centre for Professional Enhancement (CPE) at LPU operates through different specialized units that work together to improve the overall skill set of students. The technical training department focuses on enhancing coding, problem-solving, and domain-specific knowledge to make students industry-ready. Alongside this, the soft skills and communication department plays a vital role in building confidence, interpersonal abilities, and professional communication, which are essential for corporate settings. Another key department is aptitude and reasoning, which provides rigorous training in quantitative aptitude

- **Organization chart of the company**: The organizational structure of the Centre for Professional Enhancement is well-defined to ensure smooth functioning and effective delivery of training programs. At the top, the Centre is led by the Head of the Centre or Directors, who provide strategic direction and oversee all initiatives. Under them, department coordinators manage various wings such as technical training, aptitude, and soft skills, ensuring that each area is effectively supervised.

Brief Description of The Work Done

My position was a **Summer Intern**, focusing on **Data Structures and Algorithms (DSA) using C++**. My primary role was to apply theoretical knowledge by implementing and optimizing various data structures and algorithms, demonstrating proficiency in both manual implementation and the use of C++ Standard Template Library (STL).

Activities/Equipment Handled:

The main activity was the **implementation and testing of DSA concepts** in C++. This included:

- Designing and coding core data structures (arrays, linked lists, stacks, queues, trees, graphs) from scratch.
- Applying algorithms like sorting, searching, graph traversal, dynamic programming, and greedy techniques to solve problems.
- Utilizing STL components (vector, set, map, priority_queue, etc.) for efficient solutions.
- Analyzing time and space complexity for each implementation.

The primary "equipment" was my personal computer, equipped with a **C++ development environment** (Code::Blocks/Visual Studio Code,Dev c++ GCC compiler, and debugging tools). I also used online coding platforms like GeeksforGeeks and LeetCode for problem-solving practice.

Challenges Faced and How Those Were Tackled:

One key challenge was **optimizing algorithms** to handle large input sizes efficiently. This was tackled by applying **asymptotic analysis** to identify bottlenecks and replacing inefficient solutions (e.g., $O(n^2)$ algorithms) with optimized versions (e.g., $O(n \log n)$ using STL sort or improved algorithms).
Another challenge was **debugging pointer-related errors** in manual data structure implementations like linked lists and trees. This was solved by systematically tracing pointer assignments and using debugging tools to visualize memory allocation and access.

Learning Outcomes:

- Gained a **strong practical understanding** of how core data structures and algorithms work internally.
- Learned to **choose the right data structure** for a given problem based on time/space complexity trade-offs.
- Improved **problem-solving and debugging skills** in C++.
- Developed the ability to implement **optimized solutions** using both custom implementations and STL.

Data Analysis:

Data analysis involved evaluating **how different data structures store and manage data** in C++.
For example:

- Choosing between arrays and linked lists based on insertion/deletion frequency.
- Analyzing tree balancing techniques (like AVL Trees) to ensure O(log n) search times.
- Studying hashing techniques for constant-time lookups.

The goal was to design **efficient, scalable, and memory-optimized data structures** for real-world use cases.

CONCLUSION

The study and practical implementation of **Data Structures and Algorithms (DSA) using C++** provided a strong foundation in solving computational problems efficiently and systematically. This internship/project experience allowed me to move beyond theoretical concepts, focusing on how these structures and algorithms behave in real-world scenarios, particularly in terms of **time and space optimization**.

C++ was an ideal choice for learning DSA because of its:

- **Low-level control** over memory and pointers.
- **Rich Standard Template Library (STL)** for implementing efficient algorithms and data structures.
- **High performance** in competitive programming and system-level applications.

Through structured learning, I explored fundamental topics such as **algorithm analysis, arrays, linked lists, stacks, queues, trees, graphs, and hashing**, progressing to advanced areas like **dynamic programming, greedy algorithms, segment trees, and disjoint sets**. Each topic was reinforced with C++ implementations, complexity analysis, and where appropriate, visualization through graphs and diagrams.

The practical exposure included:

- **Manual implementation** of data structures to understand their internal workings.
- **Use of STL** for rapid and reliable problem-solving.
- Application of **asymptotic notations** (Big O, Ω, Θ) to assess algorithm efficiency.
- Designing data models and algorithmic solutions that balance **speed and memory usage**.
-

Challenges like optimizing algorithms for large datasets and debugging pointer errors were successfully addressed through methodical analysis, use of debugging tools, and adopting better algorithms. This process strengthened my **logical thinking, analytical ability, and programming discipline**.

Overall, this project not only solidified my theoretical understanding of DSA but also honed my **problem-solving mindset**, enabling me to select the most appropriate data structures and algorithms for different computational problems. These skills are essential for competitive

programming, technical interviews, and developing high-performance applications in real-world software engineering.

The journey from fundamentals to advanced concepts in **DSA using C++** has been transformative, equipping me with both the knowledge and the confidence to tackle complex programming challenges efficiently.

REFERENCES

https://lovelyprofessionaluniversity.codetantra.com/secure/tla/mi.jsp?s=m&m=f7010731-0250-373b-8328-901c40055fbd (Accessed on July 10, 2025)